

Semantic-Guided Fuzzing for Virtual Testing of Autonomous Driving Systems

An Guo, Yang Feng*, Yizhen Cheng, Zhenyu Chen

State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, 210093, China

Abstract

Autonomous driving systems (ADS) have achieved spectacular development and have been utilized in numerous safety-critical tasks. Nonetheless, in spite of their considerable advancement, ADS perception components with high complexity and low interpretability often demonstrate unexpected corner-case behaviors. Several real-world accidents involving self-driving cars even lead to fatalities. Before rolling the autonomous vehicles out to the end-users, it is vital to test and guarantee the safety of ADS. As one of the most critical autonomous driving testing techniques, the prevailing virtual testing depends on the tester using tool-specific languages to code traffic simulation programs correctly. However, this process often requires plenty of effort, and it may fail to capture various rare events from complex driving situations that require sophisticated awareness of the surroundings.

In this paper, we design and implement a semantic-guided scene fuzzing framework for autonomous driving systems, namely FuzzScene, based on the metamorphic testing theory. It employs driving scenario description language for scenario representation and equips a tree-based mutation strategy to generate tests with proper oracle information. To improve the testing efficiency and detect misbehaviors under different settings, we propose a unique sampling strategy and construct a testing guidance criterion to optimize FuzzScene. We experiment FuzzScene with multiple steering controllers to evaluate its performance on different tasks. The experiment results show that the semantic transformed driving scenarios generated by FuzzScene efficiently detect hundreds of inconsistent behaviors of ADS. Also, the results confirm that FuzzScene can improve steering precision by retraining with the generated scenes.

Keywords: Software Testing, Fuzz Testing, Driving Scenario Description Language, Autonomous Driving System

1. Introduction

The rapid development of artificial intelligence and sensing technology over the past decade has made autonomous vehicles around the corner. As a typical intelligent software, autonomous driving systems (ADS) have been designed and deployed to assist in many safety-critical tasks, such as transit and

delivery service [1], self-driving taxi [2], and other fields. However, such intelligent systems often exhibit incorrect and unexpected corner-case behaviors, which can cause fatal collisions when deployed in real-world environments. The National Highway Traffic Safety Administration (NHTSA) received reports of 130 crashes involving vehicles equipped with ADS from July 2021 to May 2022 [3]. Therefore, the safety of autonomous driving systems raises wide concerns, and it is imperative to conduct comprehensive testing to ensure their safety and reliability.

The primary quality assurance strategies of ADS

August 17, 2023

*Corresponding author

Email addresses: guoan218@smail.nju.edu.cn (An Guo), fengyang@nju.edu.cn (Yang Feng), yz_cheng@smail.nju.edu.cn (Yizhen Cheng), zyichen@nju.edu.cn (Zhenyu Chen)

can be categorized into two families: real-world testing and virtual testing. Real-world testing entails testing specific driving scenarios in closed autonomous vehicle proving grounds [4] or monitoring autonomous vehicles in real traffic [5], but this approach requires a long period and extensive resources. For instance, a previous study concluded that it needs a fleet of 100 autonomous vehicles to drive continuously for approximately 500 years [6]. Moreover, public road testing may fail to test against realistic variations of corner cases. On the other hand, in recent years, high-fidelity virtual testing has become essential to the development and validation of autonomous driving technologies [7, 8]. It conducts autonomous vehicle testing in simulation platforms, such as CARLA [9] and LGSVL [10]. Instead of simply calculating total driving mileage in a simulated world, we usually expose the system under test to defined driving scenarios [11, 12]. By customizing these test scenarios with varying distributions, ADS test engineers can more effectively evaluate rare and extreme scenarios and avoid wasting time on repeated scenarios. However, the ego car interacts with its diverse operating environment, including the driving path, weather conditions, and other moving agents. Simulating potential virtual crash scenarios in such an environment is a nontrivial task. Typically, key parameters that can describe a driving scenario are selected, and a parameter space is established. From this parameter space, specific scenarios with defined parameter values are extracted for testing and evaluation. Nonetheless, due to a large number of possible scenario parameters, the combinations of parameters explode when generating concrete scenarios in simulations [13]. The primary challenge is how to optimize testing efficiency. To speed up the virtual testing process, accelerated testing methods that aim to identify safety-critical scenarios are necessary.

For both conventional and DNN-based software, fuzz testing [14, 15, 16] is a widely used approach to identify bugs by navigating extensive search spaces. At a high level, fuzzing mutates existing inputs to create tests with the goal of exploring new bugs. However, integrating fuzzing into the virtual testing of ADS is not straightforward. Firstly, driving scenarios exhibit diverse characteristics and interdepen-

dencies, and random mutations of arbitrary features may result in semantically incorrect scenarios. Furthermore, ADS test engineers require additional effort to learn tool-specific languages and simulated traffic programs from scratch. Finally, these techniques often neglect corner cases or repeatedly uncover erroneous behaviors similar to those already discovered. Thus they cannot diversify the detected incorrect behaviors.

To fill the gap, we propose a systematic semantic-guided fuzzing testing approach, namely FuzzScene, for automatically testing autonomous driving systems and further retraining to improve them. FuzzScene can leverage the driving scenario description language OpenSCENARIO [17] for scenario representation. Then it parses semantic information of the driving environment and generates testing scenarios with the grammar-aware generation strategy in the CARLA simulator. Moreover, it constructs a testing guidance criterion to optimize FuzzScene and utilizes transformation-specific metamorphic relations to automatically detect diverse faults by comparing transformed driving scenes to the original data.

We conducted experiments with FuzzScene to evaluate its performance using four steering controller models. We demonstrate the efficacy of our parsed semantic transformations, as applying variable semantic transformations resulted in a decrease in Mean Squared Error (MSE). Next, our proposed learning-to-rank sampling strategy and testing criteria enable the fuzzing algorithm to discover a greater number of inconsistent behaviors than the baseline methods, and these behaviors exhibit diversity. Further, the experimental results reveal that the generated scenes can significantly increase the MSE for controller models after retraining. These results indicate that FuzzScene can effectively detect diverse inconsistent behaviors and improve the robustness of autonomous driving systems through retraining with the transformed data.

The contributions of this paper can be summarized as follows:

- **Method.** We present a semantic-guided fuzzing technique, which leverages the driving scenario description language to generate realistic and valid scenarios. Specifically, we propose a

novel grammar-aware generation strategy and design a testing guidance criterion that induces both failure and diversity to optimize FuzzScene in discovering a greater variety of inconsistent behaviors.

- **Tool.** We implement the proposed methods into an automated fuzz testing tool called FuzzScene, which enables ADS test engineers to efficiently discover various inconsistent behaviors. We have made the source code of FuzzScene available¹ and released the generated transformed driving scenes.
- **Study.** We conduct extensive studies to investigate the effectiveness of FuzzScene. We employ our tool to test four typical steering controller models and found various inconsistent decision results, many of which could lead to potentially fatal collisions.

2. Background & Motivation

2.1. Virtual Testing of ADS

Testing ADS on physical vehicles is neither cost-effective nor safe, and it is often challenging to reproduce various types of traffic accidents. A realistic and effective alternative is to test ADS under a simulated environment [18], where driving scenes and physical logic (e.g., vehicle dynamics, weather conditions, etc.) are created in a virtual world. Virtual testing allows for checking the behavior of self-driving cars in a large number of scenarios, driver characteristics, and system configurations.

To simulate traffic situations in a testing environment, various car manufacturers and organizations developed driving simulation platforms [19]. A virtual driving simulation platform needs to provide the following main components [20], including road model, traffic model, sensor model, and vehicle model. With model-based simulation, autonomous vehicles can obtain a perfect picture of the surrounding world in the driving simulator. Each simulation platform has its own characteristics; PreScan² has

a strong capability of constructing realistic environments and simulating different weather conditions. LGSVL and CARLA are suited for end-to-end testing of unique functionalities (e.g., perception, localization, etc.). Scenario description languages can assist virtual testing of autonomous driving. The scenario description files that depict the dynamic content of driving and traffic simulators are parsed and interact with the simulator to produce corresponding scenarios. ADS test engineers are only expected to create these scenario description files, as they do not need to learn the specific details of the simulator's API grammar.

Simulation of traffic scenarios is crucial to the entire verification and validation process. However, with the increased number of driving scenario key parameters, the permutations of their values explode exponentially. One core challenge is how to explore safety-critical scenarios within huge parameter space [21]. In the current literature, accelerated evaluation has been proposed to improve ADS testing efficiency. Zhang et al. [22] apply the importance sampling technique to enhance the exposure of rare events, which are considered safety-critical scenarios. Other types of testing methods identify safety-critical scenarios via step-by-step searching. The search direction is toward safety reduction and could be optimized using genetic algorithms [23] and reinforcement learning methods [24]. This type of method is compatible with black-box autonomous driving control algorithms, while it holds the issue of converging to a local optimum. By designing and choosing the most appropriate test automation methods, we can significantly optimize the efficiency and effectiveness of the entire virtual testing process.

2.2. The Driving Scenario Description Language

In the operational environment of autonomous driving, the ADS receives multi-dimensional inputs from various sensors (e.g., cameras, LiDAR, radar, etc.) and processes semantic information to drive the car. Consequently, the driving scenarios encountered by the ADS have become a meaningful metric, leading to a shift towards a scenario-oriented testing approach in the automotive industry and research community. The creation of scenarios is usually based on various data sources [25] or expert

¹<https://anonymous.4open.science/r/FuzzScene-3133>

²<https://www.plm.automation.siemens.com/global/en/products/simulation-test/active-safety-system-simulation.html>

knowledge [26], and the execution of driving scenarios can be realized in X-in-the-loop (XiL) simulations [27]. However, there requires a standard, executable, and understandable language to describe and transfer the scenarios between the creation and execution stages. To tackle this problem, both the industry and academia have been working on designing driving scenario description languages to model complex environments for ADS [28].

For describing the road layout and road features, popular open languages formats such as OpenStreetMap [29], OpenDRIVE [30], and Lanelets [31] are commonly used to form the static scenery element of driving scenarios. The ASAM OpenDRIVE format provides common road network descriptions that can be exchanged between different simulators. Lanelets is more lightweight than OpenDRIVE, yet still offers a specification for highly detailed maps. Althoff et al. [32] developed an automatic conversion tool from OpenDRIVE to the Lanelets format.

Several formal dynamic scenario description languages have been proposed for describing the dynamic contents of driving simulations at a logical level. Scenic [33] is a probabilistic programming language for scenario specification and data generation; its domain-specific design permits concise representations of complex driving scenarios and enables specialized sampling techniques. The Measurable Scenario Description Language (M-SDL) [34], introduced after the first version of Scenic, does provide declarative constraints and compositional features similar to Scenic. In contrast, GeoScenario [35], Paracosm [36] and OpenSCENARIO are not probabilistic programming languages, meaning they lack probability distributions and a declarative way of specifying geometric constraints. However, the semantic information of driving scenarios generated using non-probabilistic programming languages is more straightforward to parse and transform. OpenSCENARIO is used by various companies and organizations to create simulations and tests for autonomous vehicles [37]. In this paper, we use OpenSCENARIO for driving scenario representation and the ScenarioRunner³ module to execute traffic scenarios for the CARLA simulator.

³https://github.com/carla-simulator/scenario_runner

3. The Fuzzing Framework

In this section, we present the design and implementation of FuzzScene. We describe the design ideas and steps of the entire workflow in Section 3.2 and introduce the detailed designs in Sections 3.3, 3.4 and 3.5.

3.1. Definitions

To give a better understanding of autonomous driving virtual testing, we introduce a few terms based on previous studies [38, 39, 40]:

Scene. A scene is a snapshot in the simulation that contains the detailed properties (e.g., location, velocity, steering, etc.) of the ego car, other dynamic objects, the surrounding stationary objects, weather conditions, and road conditions. For instance, the ego car at a specific map location with a speed of 10 m/s facing south on a foggy afternoon.

Scenario. A scenario describes “the temporal development between several scenes in a sequence of scenes” [38]. Unlike a scene, a scenario spans a certain time. Two scenes may specify the same initial locations for the ego car and other moving vehicles but distinct weather conditions can result in different driving scenarios.

Logical Scenario. A logical scenario describes the parameter spaces that bound the search during the fuzzing. For example, the rain semantic transformation has its parameter range limited between 0 and 100 in this paper.

Concrete Scenario. A concrete scenario is a specific instance in the logical search space. It specifies a concrete value for each parameter of a logical scenario. Thus ADS test engineers can run specific scenarios in the simulator to reproduce test cases.

Table 1: Variable leaf nodes used by FuzzScene for generating new driving scenarios.

Semantic Transformations	Parameters	Parameter Ranges
Vehicle Color	(c_r, c_g, c_b)	$c_r \in [0, 255], c_g \in [0, 255], c_b \in [0, 255]$
Time	(t_x, t_y)	$t_x \in [1, 12], t_y \in [8, 16]$
Sun Intensity	s_i	$s_i \in [30, 100]$
Fog	v_f	$v_f \in [0, 200]$
Rain	r_n	$r_n \in [0, 100]$

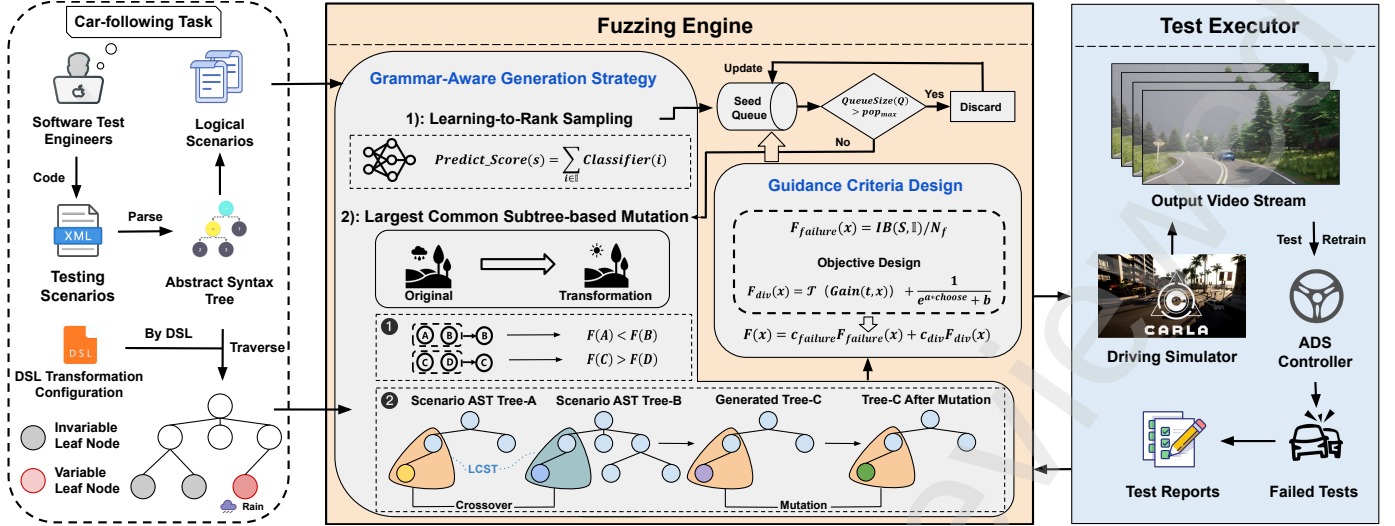


Figure 1: The Workflow of FuzzScene.

3.2. Overview of FuzzScene

In traditional fuzz testing, the fuzzer usually maintains a seed input corpus and repeatedly mutates these inputs to generate new test data. Then, after the test cases are executed, it decides whether to keep the mutated inputs based on test criteria such as code coverage. Eventually, the fuzzer terminates continuous iteration due to reaching a particular goal or a timeout.

The fuzzing technique can be applied in the autonomous driving domain. Considering the semantic transformation parameter space is prohibitively large, we propose a fuzzing framework for ADS to detect diverse inconsistent behaviors. In order to parse the environment semantics and generate syntax-valid test scenarios, we convert the driving scenario to abstract syntax trees (ASTs). Moreover, we design a testing guidance criterion to optimize FuzzScene with the aim of finding more diverse behaviors. As depicted in Fig. 1, the implementation of FuzzScene contains two major components: fuzzing engine and test executor.

ADS test engineers use scenario description language OpenSCENARIO to write testing scenarios for a specific driving task. Then testing scenarios are parsed into an AST based on the language grammar. We design logical scenarios from the AST structure and leverage them as inputs to the fuzzing engine. Meanwhile, we traverse the abstract syntax tree and obtain variable leaf nodes represent-

ing semantics according to the domain-specific language (DSL) transformation configuration file. Next, the fuzz engine efficiently leverages the grammar-aware generation strategy and guidance criteria to find combinations of semantic transformations that result in more failures. After that, the transformed video frames can be generated from CARLA to evaluate the robustness of the steering controller of autonomous driving. Finally, we achieve test reports and give feedback on semantic transformation configuration. Additionally, FuzzScene can enhance the controller’s performance by retraining with the transformed data.

Specifically, the core process of the fuzzing engine is shown in Algorithm 1. For given logical scenarios $\mathbb{L}$ and DSL transformation configuration $\mathbb{T}$, users can set predefined hyperparameters $\mathbb{C}$, generations $Gens$ and the maximum queue size pop_{max} . The fuzzing engine first adopts a learning-to-rank strategy to sample seeds from the entire input space and put them into the seed priority queue Q (Line 3). In each iteration, it selects a batch according to a seed selection method (Line 9). To generate realistic and valid concrete scenarios, the fuzzing engine generates new transformed ASTs after applying the largest common subtree-based mutation operations to the selected test cases (Line 10-19); the information on variable leaf nodes is detailed in Table 1, which contains driving semantic information such as rain and vehicle color. Then it runs simu-

lations in CARLA with the supplied scenarios. Note that the failures found are recorded, and the new tests are added to the seed queue. Finally, we update the objective values of all seeds in the queue based on the proposed objective functions (Line 21). During the whole fuzzing process, we maintain the priority queue based on objective values. The corresponding seeds with lower values are discarded if the current population is greater than pop_{max} (Line 24).

3.3. Grammar-Aware Generation Strategy

Self-driving cars rely on semantic information about the surrounding environment to form a complete picture of the driving situation. However, it can be challenging for the ego vehicle to understand scenes correctly and make the right decision in real-time. Even slight variations in traffic situations, such as changing an object’s color, can potentially result in various traffic violations being reported. To be explicitly aware of the grammar and thus produce syntax-valid test inputs, we utilize the grammar knowledge and design a grammar-aware generation strategy, which works at the level of ASTs. AST actually models a test input as elements with named attributes and is designed to represent all the information about a test input. Thus, ASTs provide a suitable granularity for a fuzzer to generate test inputs.

3.3.1. Learning-to-Rank Sampling

In order to automatically find suitable initial attribute values of ASTs when sampling seed test cases, we propose a learning-to-rank sampling strategy for fuzz testing to uncover more inconsistent behaviors.

Our approach constructs a training set capturing data from CARLA for learning-to-rank, where each instance is an input of the steering model under evaluation. For each instance, we label it as 0 or 1 depending on whether the input is incorrectly predicted by the model under test. Then we train a binary classifier for each logical scenario to minimize the cross-entropy loss. Given a sampling seed s , the corresponding classifier predicts whether each intercepted video frame will fail or not. For the image stream $\mathbb{I}$ captured from scenario s , we accumulate the model

outputs as the prediction score, which is defined as:

$$Predict_Score(s) = \sum_{i \in \mathbb{I}} Classifier(i) \quad (1)$$

In the learning-to-rank sampling process, we randomly instantiate gen_m candidate seed test cases for each logical scenario. Subsequently, our learning-to-rank strategy prioritizes gen_m candidate seeds according to their descending scores and selects the top q_r seeds. Finally, we save all selected seeds into the seed queue.

3.3.2. Largest Common Subtree-based Mutation

As the number of iterations increases, it is necessary to apply some seed data selection and mutation strategies to sample seed data for transformations. In this paper, we design a tree-based mutation algorithm to be explicitly aware of the grammar to generate syntax-valid scenarios. Specifically, our proposed largest common subtree-based (LCST-based) mutation strategy facilitates the incorporation of similar semantic information from two driving scenarios.

First, we use the binary tournament selection method [41]. This method randomly samples two parent candidate seeds from the current seed queue of size N into a tournament, and the winner is chosen based on their objective function values. We repeat this process k times to select k winners. The winners are then paired up to serve as the selected parents for the following crossover and mutation steps. Then we parse each paired winner into an AST. Using ASTs, we introduce the LCST-based mutation strategy that can effectively mutate test inputs while keeping the input structure valid. The LCST problem is to find a common subtree (based on a bijective mapping) with the maximum number of nodes [42].

Algorithm 2 shows the procedure of our tree-based mutation. It takes as inputs test scenario description files A , B , and the grammar G . It first parses test inputs A and B according to G into ASTs (Line 2–3). After copying the ASTs (Line 4–5), it computes the largest common subtree of the two ASTs. We store each obtained largest common subtree res_tree in the set $\mathbb{P}$, and then trim T_A and T_B according to the res_tree structure until T_A and T_B have no largest common subtree (Line 7–11). Next, for each subtree

s in $\mathbb{P}$, A_{tree} and B_{tree} are individually matched with the structure of s, and subsequently, the variable leaf nodes of the matched structures in A_{tree} and B_{tree} are paired (Line 12– 14). For each pair of leaf nodes matched by the two ASTs, we adopt the simulated binary crossover (SBX) [43] to simulate the working principle of the single-point crossover operator on binary strings. We set a suitable distribution index η_d and probability p_c to promote the diversity of the test inputs. Further, we apply the polynomial mutation [44] to each value for leaf nodes with a pre-specified mutation rate m_r and the mutation magnitude η_m to promote mutations. Finally, it converts ASTs to mutated scenario description files m_A and m_B (Line 15).

Algorithm 1: Semantic-guided scene fuzzing

Input : the tested model S , DSL transformation configuration $\mathbb{T}$, designed logical scenarios $\mathbb{L}$, predefined hyperparameters $\mathbb{C}$, maximum queue size pop_{max} , generations $Gens$

Output: diverse inconsistent behaviors $\mathbb{D}$

```

1  $Q \leftarrow \emptyset$ ;
2 InitializeFuzzer( $\mathbb{T}, \mathbb{C}, S, pop_{max}, Gens$ );
3  $seeds \leftarrow \text{LearningtoRankSampling}(\mathbb{L}, \mathbb{T})$ ;
4  $Q.append(seeds)$ ;
5 foreach  $seed \in Q$  do
6   InitSeed( $seed$ );
7 end
8 for  $iterate \leftarrow 1$  to  $Gens$  do
9    $selected\_seeds \leftarrow \text{TournamentSelection}(Q)$ ;
10   $i \leftarrow 1$ ;
11  while  $i \leq \text{len}(selected\_seeds)$  do
12     $pop_a, pop_b \leftarrow \text{LCSTMutation}(t_i, t_{i+1})$ ;
13     $Q.append(pop_a)$ ;
14     $Q.append(pop_b)$ ;
15     $i \leftarrow i + 2$ ;
16  end
17  foreach  $seed \in Q$  do
18    UpdateObj( $seed$ );
19  end
20  if  $\text{QueueSize}(Q) > pop_{max}$  then
21    UpdateQueue( $Q$ );
22    // maintain queue size
23  end
24 return  $\mathbb{D}$ 

```

3.4. Guidance Criteria Design

For conventional software programs, various structural code coverage is often employed as the guidance criteria in the fuzzing process. Different from traditional software, modern ADS contain multiple deep learning algorithms and models, which

rely on a large amount of labeled data to predict or classify new input data after training. The particularity of ADS construction makes the traditional code coverage method ineffective, which is confirmed in many studies of testing DNN models and DNN-based software [45, 46]. To overcome this challenge and ensure testing efficiency, we incorporate failure-inducing and diversity-inducing properties, then design testing criteria as guidance in the test generation process.

For the failure-inducing property, we design an objective $F_{failure}$ to measure inconsistent behaviors during a simulation and maximize it. Given a concrete scenario x , the objective can be denoted as follows:

$$F_{failure}(x) = IB(S, \mathbb{I})/N_f \quad (2)$$

where N_f represents the total number of video frames we collected from concrete scenario x , $IB(S, \mathbb{I})$ represents the number of inconsistent behaviors of a concrete autonomous driving scenario, which is detailed in Section 4.3.

For the diversity-inducing property, we take an information-theoretic perspective and develop an entropy-based objective function. This objective function assigns more energy to seeds that promote diversity. To further discover new error categories, we attempt to fuzz different seeds in each iteration. Next, we present more details of our design.

In the fuzzing campaign, we reserve a $L_s \times N_f$ probability matrix $\mathbf{P} = (p_{ij})$ for each iteration, which can be represented as:

$$\mathbf{P} = \begin{pmatrix} p_{11} & \cdots & p_{1N_f} \\ \vdots & \ddots & \vdots \\ p_{L_s 1} & \cdots & p_{L_s N_f} \end{pmatrix} \quad (3)$$

where L_s is the total number of logical scenarios, then we compute entropy $H(t)$ of the current iteration t based on Shannon's entropy [47] as follows:

$$H(t) = - \sum_{i=1}^{L_s} \sum_{j=1}^{N_f} p_{ij} \log(p_{ij}) \quad (4)$$

where p_{ij} is the probability that failure belongs to the j th frame of i th logical scenario. To calculate the conditional information entropy after simulating sce-

nario x in the current state t , we define $H(x|t)$, which can be computed as:

$$H(x|t) = - \sum_{i=1}^{L_s} \sum_{j=1}^{N_f} p'_{ij} \log(p'_{ij}) \quad (5)$$

In the above equation, p'_{ij} is the changed probability that failure belongs to the j th frame of i th logical scenario. We estimate entropy H based on how often we have seen each error category. Therefore, we calculate a $L_s \times N_f$ total error frequency matrix $Y = (y_{ij})$ for each iteration as follows:

$$Y = \begin{pmatrix} y_{11} & \cdots & y_{1N_f} \\ \vdots & \ddots & \vdots \\ y_{L_s 1} & \cdots & y_{L_s N_f} \end{pmatrix} \quad (6)$$

To ensure $p_{ij} \neq 0$, the value of each element in the matrix is set to one before iteration takes place. During the iteration process, if the j th frame of the i th logical scenario experiences an error, the frequency value of the corresponding position is incremented by one. Then an estimator of the error discovery probability p_{ij} is $\hat{p}_{ij} = y_{ij}/total_{error}$, where y_{ij} the frequency of failures, $total_{error}$ is the sum of failure frequencies in Y when iterating up to t rounds.

Similar to p_{ij} , an estimator of p'_{ij} is $\hat{p}'_{ij} = y'_{ij}/total'_{error}$. Note $y'_{ij} = y_{ij} + 1$ when a specific instance x of i th logical scenario occurs an error at the j th frame. Compared to $total_{error}$, $total'_{error}$ additionally includes the errors found in the concrete scenario x .

By substituting $\hat{p}_{ij}$ and $\hat{p}'_{ij}$ into Equation (4) and (5) respectively, we can obtain the estimated entropy $\hat{H}_E(t)$ and $\hat{H}_E(t|x)$. Using these estimates, we are capable of calculating the information gain for x at iteration t , i.e. $Gain(t, x) = \hat{H}_E(x|t) - \hat{H}_E(t)$. Additionally, we track the number of times each seed has been fuzzed to select different logical scenarios. Finally, we express the diversity objective F_{div} for a concrete scenario x as:

$$F_{div}(x) = \mathcal{T}(Gain(t, x)) + \frac{1}{e^{a*choose+b}} \quad (7)$$

where $\mathcal{T}$ normalizes the information gain values of the seeds in the seed queue to the range $[0, 1]$. a , b are coefficients, and $choose$ represents the number of times the seed has been fuzzed. Putting the above

objectives together, we obtain the following criteria that our fuzzer attempts to optimize:

$$F(x) = c_{failure} F_{failure}(x) + c_{div} F_{div}(x) \quad (8)$$

where $c_{failure}$ and c_{div} are coefficients that are used to weigh the impact of the failure and diversity objectives, respectively. After performing generation operations in each iteration, the fuzzer updates the testing criteria for each seed in the queue.

Algorithm 2: LCST-based Mutation

Input : the test input A , the grammar G , the test input B
Output: mutated test input m_A , mutated test input m_B

```

1  $\mathbb{P} \leftarrow \emptyset$ ;
2 parse  $A$  according to  $G$  into an AST  $A_{tree}$ 
3 parse  $B$  according to  $G$  into an AST  $B_{tree}$ 
4  $T_A \leftarrow \text{Copy}(A_{tree})$ ;
5  $T_B \leftarrow \text{Copy}(B_{tree})$ ;
6  $res\_tree \leftarrow \text{LCST}(A_{tree}, B_{tree})$ ;
7 while  $res\_tree$  is not empty do
8    $\mathbb{P} \leftarrow \mathbb{P} \cup \{res\_tree\}$ ;
9    $T_A, T_B \leftarrow \text{TrimTree}(T_A, T_B, res\_tree)$ ;
10   $res\_tree \leftarrow \text{LCST}(T_A, T_B)$ ;
11 end
12 foreach subtree  $s \in \mathbb{P}$  do
13    $A_{tree}, B_{tree} \leftarrow \text{CrossMutation}(A_{tree}, B_{tree}, s)$ ;
14 end
15  $m_A, m_B \leftarrow \text{ConvertFile}(A_{tree}, B_{tree})$ ;

```

3.5. Test Oracle Construction

A complete ADS must include braking, acceleration, and steering controllers. One of the significant challenges in testing a single controller is to create the system's specifications manually, as it essentially involves recreating the logic of human drivers [48, 49, 50]. To avoid this issue, we leverage metamorphic relations (MRs) [51] between the car behaviors across the original image and its transformed image. The core idea is that we can define relationships between the vehicle's behaviors across certain types of transformations. For instance, the autonomous car's steering angle should not change hugely for the same image under any semantic transformation. The violation of MRs often reveals potential defects.

Our approach is designed to realize the semantic MRs for ADS. Specifically, we denote the steering controllers of autonomous driving as $\mathbb{S}$, which predicts the car's steering behaviors. The MRs can be defined as given the original image stream $\mathbb{I}$, and semantic transformations $\gamma \in \mathbb{T}$ can generate various

mutation images from each image $i \in \mathbb{I}$. The semantic MR to test $\mathbb{S}$ with additional transformed images can be formalized as:

$$\forall i \in \mathbb{I} \wedge \forall \gamma \in \mathbb{T}, \zeta\{S[\gamma(i)], S[i]\} \quad (9)$$

where ζ is a criterion asserting the equality of $\mathbb{S}$ steering outputs. The given MR is defined such that regardless of how the image is synthesized by applying semantic transformation γ on i , the steering angle is expected to be consistent with that of the original image.

However, asserting the equality of $\mathbb{S}$ outputs is too strict, as there is usually no single correct steering angle for a given image. This approach may result in many false positives. In order to make the MRs more permissive, we allow variations within the error ranges of the original input scenarios. We define the number of inconsistent behaviors of ADS in Section 4.3. Given semantic MRs, we can simply check the test oracle whether it is satisfied in the testing process of $\mathbb{S}$.

4. Experiment Design

In this section, we introduce our experimental setup, including the dataset and steering models under-tested, parameter settings, evaluation metrics, and the research questions we study. To conduct the experiments, we implement the fuzzing framework upon TensorFlow 2.5.0 and use CARLA 0.9.13 as the simulator. All experiments are performed on a Ubuntu 21.10 desktop with GeForce RTX 3070, one 16-core processor with 3.80GHz and 32GB physical memory.

FuzzScene is designed to systematically test the autonomous driving system via semantic-guided fuzzing. To this end, we empirically explore the following four research questions (RQ):

- **RQ1:** Can FuzzScene generate test suites triggering more faults in comparison to baselines?
- **RQ2:** How diverse are the faults generated by FuzzScene?
- **RQ3:** Can we leverage erroneous behaviors found by FuzzScene to enhance the steering controllers?

4.1. Dataset and Steering Controllers

Dataset. For both training and testing datasets, we capture and collect a large number of video frames using the camera mounted on the ego vehicle from multiple CARLA route maps. The corresponding ground truth steering angles are acquired from the control signals, where negative steering angles indicate left turns, and positive values denote right turns. The data collected and generated in this paper has a maximum steering angle of +/- 25 degrees. To ensure that the predicted steering angles fall within the range of -1 and 1, the steering angle is scaled by a factor of 1/25.

Controllers. To evaluate the performance of our techniques effectively, we leverage four popular steering controller models, which have been widely used in autonomous driving testing [52, 48]. Specifically, we test three models based on the Dave self-driving car architecture from NVIDIA with different configurations, denoted as Dave-orig [53], Dave-norminit [54], Dave-dropout [55], as well as the Epoch model from the Udacity challenge [56]:

- **Dave-orig** fully replicates the original architecture presented in NVIDIA’s Dave system.
- **Dave-norminit** removes the first batch normalization layer and normalizes the initialized network weights.
- **Dave-dropout** is another publicly available steering model which modifies the original Dave model by cutting down the number of convolutional layers and fully connected layers and inserting dropout layers among the fully connected layers.
- **Epoch** model consists of a single convolutional neural network that achieves a high rank in the Udacity self-driving challenge.

In order to determine whether FuzzScene is an effective tool for testing ADS, we obtain pre-trained steering models by training them with the collected image frames. Note that in original testing scenarios, Dave-orig, Dave-norminit, Dave-dropout, and Epoch achieved 0.017, 0.016, 0.028, and 0.026 MSE, respectively, which indicate they performed well.

4.2. Scenario and Parameter Settings

Scenario Select. We select six logical scenarios (i.e. $L_s = 6$) from various CARLA route maps, including Town04, Town06, and Town07. These logical scenarios inspired by the NHTSA report mainly describe the ego car following a leading vehicle. To ensure safe driving on the road, the ego car needs to make correct steering decisions for each scene. We capture corresponding frames when FuzzScene runs a concrete scenario, and we capture enough video frames ($N_f=125$) in our experiment design.

Parameter Settings. FuzzScene focuses on finding inconsistent behaviors of ADS by creating realistic semantic transformations. For the learning-to-rank sampling process, the hyper-parameters gen_m and q_r are set to 12 and 4. We set the selection times k for each generation to 4. In the crossover & mutation step, we configure η_d , p_c , m_r , and η_m to be 10, 0.5, 0.05, and 5, respectively, to promote diversity in the offspring. We set the default values for $c_{failure}$, c_{div} , a , b in the objective function (defined in Section 3.4) to 1, 1, 1, 0. The default values control the contribution of the failure-inducing property ($c_{failure}$) and the diversity-inducing property (c_{div}). The positive signs of $c_{failure}$ and c_{div} signify the aim to maximize $F_{failure}$ and F_{div} . The fuzzing settings include 25 generations with four simulations per generation in the experiment, and the maximum size of the seed queue is 50.

4.3. Metric and Baseline Comparison

Metric. We employ the MSE to measure the testing performance of FuzzScene and answer research questions in this paper. And we express it as follows:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{\theta}_{gi} - \theta_{mi})^2 \quad (10)$$

where $\hat{\theta}_{gi}$ represents the ground truth steering label of frame i , θ_{mi} is the respective output predicted by the steering model and n is the number of frames.

In this research, an autonomous driving system is defined to act consistently if its steering angle prediction falls within certain error bounds after modifying the semantic information of driving scenarios [49].

We represent the number of inconsistent behaviors of a concrete autonomous driving scenario as follows:

$$IB(S, \mathbb{I}) = \sum_{i \in \mathbb{I}} f((S[i] - S[\gamma(i)])^2 > \lambda MSE_{orig}) \quad (11)$$

where $\mathbb{S}$ denotes the autonomous driving model, and $\mathbb{I}$ is the image stream of the driving scenario, i denotes the i th frame in $\mathbb{I}$. γ denotes the transformer which can change the semantic information of the original image. f is an indicator function that outputs 1 or 0 if and only if the input is true or false, MSE_{orig} is the mean squared error of the original scenario, and λ is a configurable parameter set to be 1 in experiments.

In order to reflect the diversity of faults found by FuzzScene, we use the concept of error category. The total error categories of logical scenario $l \in \mathbb{L}$ we found are defined as:

$$Error_Category(l) = \sum_{i=1}^{N_l} \mathbb{1} [\exists x_l \text{ s.t. } IB(S, i) = 1] \quad (12)$$

where x_l is a concrete scenario instantiated from logical scenario l , N_l is the number of scenes (frames) instantiated by l , and $\mathbb{1}$ is a mapping of a condition truth value to a numeric value in $\{0, 1\}$.

Baseline. We noticed that several research works are proposed to generate tests for ADS. However, unlike these works, our technique handles diverse logical scenarios during the fuzzing process to test the steering controller component and satisfies the metamorphic relations when running scenarios.

- To evaluate the effectiveness of FuzzScene in finding erroneous behaviors, we use three baselines: random search (RANDOM), random sampling (RD-SAM), and the fuzz algorithm with objective F_{div} (FA-DIV). The RA-SAM method generates the initial seeds randomly, and the FA-DIV method runs the fuzzer without $F_{failure}$ in the objective function.
- To assess the ability of FuzzScene to detect diverse faults, we leverage the fuzz algorithm with $F_{failure}$ (FA-FAIL) as the baseline. The FA-FAIL method runs the fuzzer without F_{div} in the objective function.

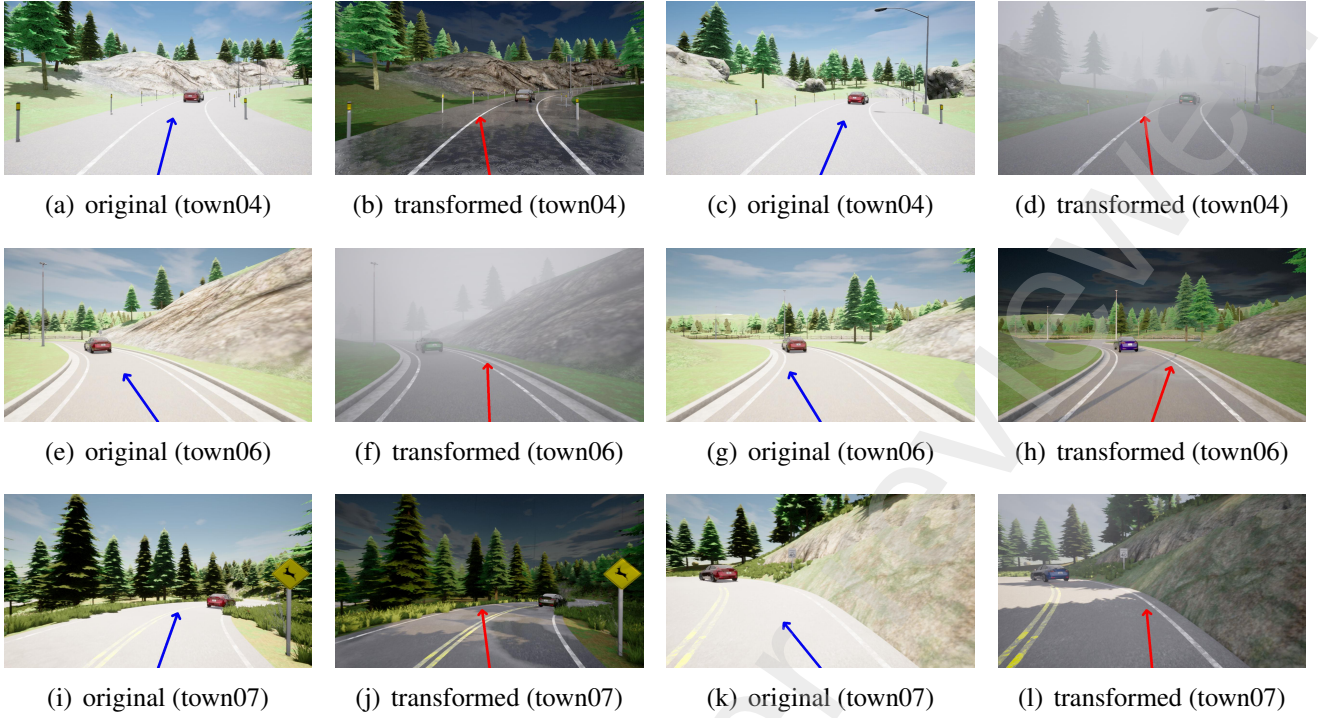


Figure 2: Visualization samples showing inconsistent behaviors detected by FuzzScene using transformed scenes. For original scenes the arrows are marked in blue, while for the transformed scenes they are marked in red.

5. Results, Analysis and Discussion

This section shows the experiment result and then analyzes the performance of our approach.

5.1. Answer to RQ1

To evaluate the fault detection ability of FuzzScene, we explore whether it can find more realistic inconsistent behaviors. We compare it with the three baselines described in Section 4.3. Note that all the inconsistent behaviors are generated by valid semantic scenarios, as they are transformed using our grammar-aware generation strategy.

Results. Fig. 3 shows the number of inconsistent behaviors found by the four methods for each controller. FuzzScene found 31%, 9%, 31%, and 29% more misbehaviors than the best baseline method under each configuration, respectively. As the iteration number increases, the FuzzScene framework discovers significantly more inconsistent behaviors than the RANDOM method.

To further prove the effectiveness of FuzzScene in detecting inconsistent behaviors, we present six examples of such behaviors intercepted by FuzzScene, as illustrated in Fig. 2. These inconsis-

tencies are discovered across various CARLA route maps (Town04, Town06, and Town07). Although only minor semantic modifications are added to the original scene, the steering angle predicted by the controller model is changed substantially.

Discussion. After analysis, our technique is capable of identifying various combinations of transformations, which represent different environmental semantics (e.g., rain intensity, vehicle color, etc.) and lead to potential failures, according to Fig. 2. Additionally, Fig. 3 shows that FuzzScene, FA-DIV, and RD-SAM find more inconsistent behaviors than the RANDOM method for all configurations, thereby indicating that our designed fuzzing framework is a productive approach for identifying bugs by navigating large search spaces. Moreover, the proposed learning-to-rank sampling and $F_{failure}$ are effective as the number of detected defects increased substantially after implementing the failure-inducing objective and learning-to-rank sampling strategy.

5.2. Answer to RQ2

As failure-inducing inputs usually are very dense and close to each another [57, 58], similar faults

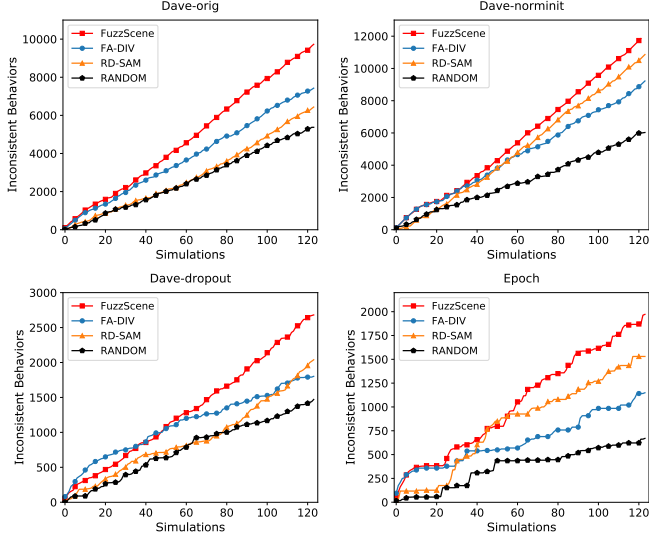


Figure 3: The number of inconsistent behaviors found on the steering controllers over simulations.

may reflect the same defect. Therefore, to comprehensively analyze the steering controller, we aim to detect more inconsistent behaviors and find diverse faults. In RQ3, we analyze the diversity of faults generated by FuzzScene, i.e., whether generated test scenarios can trigger distinct error categories. We compare FuzzScene with FA-FAIL using the error category metric described in Section 4.3. Further, we classify each error according to the logical scenario and calculate the Gini impurity [46] metric to show the impurity of inconsistent behaviors.

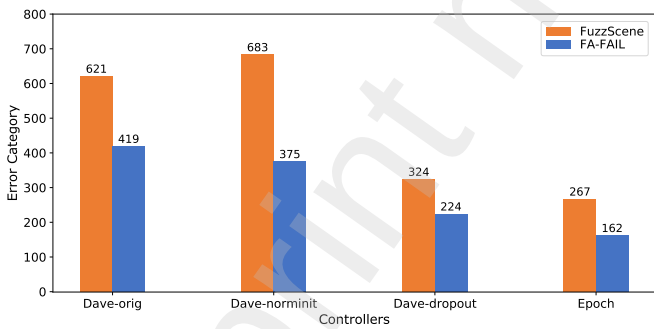


Figure 4: The number of error categories found by fuzzers with different configurations over simulations.

Results. Fig. 4 presents the number of error categories detected by FuzzScene and FA-FAIL. FuzzScene found 48.2%, 82.1%, 44.6%, and 64.8% more error categories than the baseline method for each configuration, respectively. Table 2 further

Table 2: Error category impurity measured by the Gini.

Controller	FA-FAIL	FuzzScene
Dave-orig	0.19	0.83
Dave-norminit	0.60	0.80
Dave-dropout	0.25	0.82
Epoch	0.30	0.82

demonstrates the diversity of generated scenes measured by Gini impurity. We can observe that FuzzScene with both F_{div} and $F_{failure}$ objective functions as test generation guidance can trigger more error categories and achieve higher Gini impurity for all settings.

Table 3: The test result comparison after retraining the controller with the transformed data generated by FuzzScene.

Controller	Original MSE	Retrained MSE	Fixed Scenes
Dave-orig	0.097	0.031	580/900
Dave-norminit	0.192	0.027	577/900
Dave-dropout	0.080	0.026	882/900
Epoch	0.138	0.052	851/900

Discussion. We calculate the number of triggering error categories and the impurity of the inconsistent behaviors to show the diversity of faults found by FuzzScene. Intuitively, the Gini value reflects the probability that two samples randomly selected from erroneous behaviors belong to different logical scenarios. Therefore, a higher Gini value indicates greater impurity of the faults, i.e., the more diverse the inconsistent behaviors. Furthermore, considering errors belonging to specific categories are unlikely to be generated, even with an exhaustive search, the theoretical maximum number of error categories (i.e., 750) cannot always be achieved. Despite this limitation, our framework is able to trigger a large number of error categories and achieve high Gini values. The experimental results show that the proposed diversity-inducing objective, F_{div} , is effective in increasing the fault diversity.

5.3. Answer to RQ3

Here we examine whether retraining the steering controllers with the inconsistent behaviors generated

by FuzzScene helps improve their performance. To reduce the time cost associated with retraining, we randomly select 1800 detected inconsistent behaviors by FuzzScene in RQ2 for each controller and split the corresponding failure frames into 900 for retraining and 900 for testing. Subsequently, we evaluate the retrained steering model on the held-out 900 previously failing scenes.

Results. Table 3 demonstrates the optimization effect and the number of fixed scenes for all controllers after retraining with the inconsistent behaviors generated by FuzzScene. We compute the MSE decrease and scene repair rate for each controller. The MSE reduction percentages for Dave-orig, Dave-norminit, Dave-dropout, and Epoch are 68%, 86%, 68%, and 62% respectively. The repair rates for Dave-orig, Dave-norminit, Dave-dropout, and Epoch are 64%, 64%, 98%, and 95% respectively.

Discussion. Our experimental results show that the retrained controller succeeds in at least 64% of the originally failing scenes. Therefore, using a small number of erroneous behaviors generated by FuzzScene for retraining can significantly improve the performance of the controller model on the car-following task. Furthermore, the degree of improvement is correlated with the original steering precision. The experimental results show that relatively high steering precision may limit the room for improvement of the controller model.

5.4. Threats to Validity

Data Simulation. One of the primary threats to validity is data simulation. Our approach capture and collect the driving scenes from the open-source simulator for autonomous driving research. The simulated driving scenes may differ visually and physically from the real world because of the gap that exists between testing in the physical world and testing in a simulator. To alleviate this threat, we use the popular high-fidelity CARLA [9] simulator, which has been built for flexibility and realism in both rendering and physics simulation.

The Driving Task Selection. In this paper, we limit our evaluation to the car-following task in testing autonomous driving vehicles due to the complexity of its applicable environments. Other driving behaviors, including cut-in and cut-out, etc., may also

lead to erroneous decisions of the ADS. It is difficult to ensure the efficacy of FuzzScene in improving safety for every driving task. However, as car-following [59] is one of the most fundamental behaviors in autonomous driving, our fuzzing framework shows promising potential for use in testing other driving behaviors as well.

Parameter Settings. Another threat could be caused by the configurable parameters, as the hyperparameters (e.g., the coefficients of the objective functions) are not fully fine-tuned. Nevertheless, even with the current parameters, our approach has already outperformed the baselines with four steering models (Dave-orig, Dave-norminit, Dave-dropout, and Epoch). By fine-tuning the hyperparameters, the performance of our proposed method can be further optimized.

6. Related Work

In this section, we primarily introduce the related works on two aspects: testing approaches of autonomous driving systems and metamorphic testing.

6.1. Testing of Autonomous Driving Systems

Automatically constructing and exploring diverse and challenging driving scenes is of great significance for the virtual testing of autonomous driving systems, due to the numerous possible traffic scenarios and their complexity [60, 61]. A recent set of approaches [48, 49, 62] leverages metamorphic testing techniques to examine inputs that trigger inconsistencies between the original and transformed driving scenes. Additionally, existing accelerated evaluation methods concentrate on optimizing virtual testing and improving efficiency [63, 64]. Gambi Alessio et al. [65] generate effective simulation-based tests automatically by combining a genetic algorithm and procedural content generation and expose many safety-critical failures related to lane-keeping functionality of autonomous vehicles. Li et al. [15] propose an automated fuzzing framework that can generate safety violation scenarios for self-driving cars. Kim et al. [66] present DriveFuzz, an end-to-end fuzzing framework designed to find bugs ADSes that are readily exploitable by attackers.

The key differences between the related work mentioned above and FuzzScene are threefold: (1) ADS test engineers require to make an extra effort to learn tool-specific languages and simulated traffic programs, which is a laborious and time-consuming task, while our testing and enhancement approach employs driving scenario description language OpenSCENARIO to alleviate this problem; (2) FuzzScene generates variations of corner scenarios with a grammar-aware generation strategy and designs metamorphic relations to test single steering controller components for car-following tasks. In contrast, related studies do not consider parsing the scenario into an abstract syntax tree for semantic transformations and do not test for specific driving tasks; and (3) We design a guidance criterion to generate test suites with sufficient diversity and fault detection ability in FuzzScene, whereas the aforementioned approaches only consider finding safety violations in a relatively short period of search time;

6.2. Fuzz Testing

Fuzz testing (a.k.a. fuzzing) is an automatic and effective software testing technique that can discover both correctness and security bugs [67]. During the fuzzing process, the basic way to reveal hidden bugs is to repeatedly produce numerous suitable test cases for target programs and monitor the program's exceptions. Fuzz testing has been used to detect vulnerabilities in both traditional software [68] and deep learning applications [69], and it tends to work well with relatively simple input formats such as audio or image. For more complex input formats such as protocol messages [70] or cloud service APIs [71], researchers usually employ grammar-based fuzzing [72] to obey domain-specific constraints and narrow down the search space for generating effective and valid inputs.

In this paper, we develop a semantic-guided fuzzing framework for the virtual testing of ADS. Specifically, our approach fuzzes the constrained input space by leveraging the grammar-aware generation strategy. In addition, we design a testing guidance criterion to accelerate the search for safety-critical driving scenarios.

7. Conclusion

In this paper, we present FuzzScene, a semantic-guided fuzzing technique for finding inconsistent behaviors of ADS during virtual testing. FuzzScene leverages the driving scenario description language OpenSCENARIO to describe scenarios. It innovatively designs a grammar-aware generation strategy and performs a guidance criterion for efficiently finding combinations of semantic transformations that result in more failures. FuzzScene has been evaluated on four steering controllers. The experiment results demonstrate that the transformed data generated by FuzzScene can effectively detect diverse inconsistent behaviors and improve controller model performance via retraining.

Acknowledgments

The authors would like to thank the anonymous reviewers for their insightful comments. This project was partially funded by the National Natural Science Foundation of China under Grant Nos. 61932012.

References

- [1] Japan approves use of highly autonomous 'level-4' self-driving vehicles from april '23, <https://www.japantimes.co.jp/news/2022/12/20/business/level-4-highly-autonomous-self-driving-vehicles/> (2022).
- [2] Baidu's self-driving taxis now a common sight in beijing, <https://asia.nikkei.com/Business/China-tech/Baidu-s-self-driving-taxis-now-a-common-sight-in-Beijing> (2022).
- [3] Us releases new driver-assist crash data, and surprise, it's mostly tesla, <https://www.theverge.com/2022/6/15/23168088/nhtsa-adas-self-driving-crash-data-tesla> (2022).
- [4] G. Lou, Y. Deng, X. Zheng, M. Zhang, T. Zhang, Testing of autonomous driving systems: where are we and where should we go?, in: A. Roychoudhury, C. Cadar, M. Kim (Eds.), Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022, ACM, 2022, pp. 31–43. doi:10.1145/3540250.3549111.
- [5] X. Zhao, V. Robu, D. Flynn, K. Salako, L. Strigini, Assessing the safety and reliability of autonomous vehicles from road testing, in: K. Wolter, I. Schieferdecker,

- B. Gallina, M. Cukier, R. Natella, N. R. Ivaki, N. Laranjeiro (Eds.), 30th IEEE International Symposium on Software Reliability Engineering, ISSRE 2019, Berlin, Germany, October 28-31, 2019, IEEE, 2019, pp. 13–23. doi:10.1109/ISSRE.2019.00012.
- [6] N. Kalra, S. M. Paddock, Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability?, *Transportation Research Part A: Policy and Practice* 94 (2016) 182–193.
- [7] W. Li, C. Pan, R. Zhang, J. Ren, Y. Ma, J. Fang, F. Yan, Q. Geng, X. Huang, H. Gong, W. Xu, G. Wang, D. Manocha, R. Yang, AADS: augmented autonomous driving simulation using data-driven algorithms, *Sci. Robotics* 4 (28) (2019). doi:10.1126/scirobotics.aaw0863.
- [8] Y. Abeysirigoonawardena, F. Shkurti, G. Dudek, Generating adversarial driving scenarios in high-fidelity simulators, in: *International Conference on Robotics and Automation, ICRA 2019, Montreal, QC, Canada, May 20-24, 2019, IEEE, 2019, pp. 8271–8277. doi:10.1109/ICRA.2019.8793740.*
- [9] A. Dosovitskiy, G. Ros, F. Codevilla, A. M. López, V. Koltun, CARLA: an open urban driving simulator, in: *1st Annual Conference on Robot Learning, CoRL 2017, Mountain View, California, USA, November 13-15, 2017, Proceedings, Vol. 78 of Proceedings of Machine Learning Research, PMLR, 2017, pp. 1–16.*
- [10] G. Rong, B. H. Shin, H. Tabatabaee, Q. Lu, S. Lemke, M. Mozeiko, E. Boise, G. Uhm, M. Gerow, S. Mehta, E. Agafonov, T. H. Kim, E. Sterner, K. Ushiroda, M. Reyes, D. Zelenkovsky, S. Kim, LGSVL simulator: A high fidelity simulator for autonomous driving, in: *23rd IEEE International Conference on Intelligent Transportation Systems, ITSC 2020, Rhodes, Greece, September 20-23, 2020, IEEE, 2020, pp. 1–6. doi:10.1109/ITSC45102.2020.9294422.*
- [11] P. Junietz, W. Wachenfeld, K. Klonecki, H. Winner, Evaluation of different approaches to address safety validation of automated driving, in: W. Zhang, A. M. Bayen, J. J. S. Medina, M. J. Barth (Eds.), *21st International Conference on Intelligent Transportation Systems, ITSC 2018, Maui, HI, USA, November 4-7, 2018, IEEE, 2018, pp. 491–496. doi:10.1109/ITSC.2018.8569959.*
- [12] J. Sun, H. Zhang, H. Zhou, R. Yu, Y. Tian, Scenario-based test automation for highly automated vehicles: A review and paving the way for systematic safety assurance, *IEEE Trans. Intell. Transp. Syst.* 23 (9) (2022) 14088–14103. doi:10.1109/TITS.2021.3136353.
- [13] F. Gao, J. Duan, Y. He, Z. Wang, A test scenario automatic generation strategy for intelligent driving systems, *Mathematical Problems in Engineering* 2019 (2019). doi:10.1155/2019/3737486.
- [14] B. P. Miller, M. Zhang, E. R. Heymann, The relevance of classic fuzz testing: Have we solved this one?, *IEEE Trans. Software Eng.* 48 (6) (2022) 2028–2039. doi:10.1109/TSE.2020.3047766.
- [15] G. Li, Y. Li, S. Jha, T. Tsai, M. B. Sullivan, S. K. S. Hari, Z. Kalbarczyk, R. K. Iyer, AV-FUZZER: finding safety violations in autonomous driving systems, in: M. Vieira, H. Madeira, N. Antunes, Z. Zheng (Eds.), *31st IEEE International Symposium on Software Reliability Engineering, ISSRE 2020, Coimbra, Portugal, October 12-15, 2020, IEEE, 2020, pp. 25–36. doi:10.1109/ISSRE5003.2020.00012.*
- [16] H. B. Braiek, F. Khomh, On testing machine learning programs, *Journal of Systems and Software* 164 (2020) 110542.
- [17] Asam, <https://www.asam.net/standards/detail/openscenario> (2020).
- [18] Q. Chao, H. Bi, W. Li, T. Mao, Z. Wang, M. C. Lin, Z. Deng, A survey on visual traffic simulation: Models, evaluations, and applications in autonomous driving, *Comput. Graph. Forum* 39 (1) (2020) 287–308. doi:10.1111/cgf.13803.
- [19] P. Kaur, S. Taghavi, Z. Tian, W. Shi, A survey on simulators for testing self-driving cars, in: *2021 Fourth International Conference on Connected and Autonomous Driving (MetroCAD), IEEE, 2021, pp. 62–70. doi:10.1109/MetroCAD51599.2021.00018.*
- [20] H.-P. Schöner, Simulation in development and testing of autonomous vehicles, in: *18. Internationales Stuttgarter Symposium, Springer, 2018, pp. 1083–1095. doi:10.1007/978-3-658-21194-3_82.*
- [21] Y. Wang, R. Yu, S. Qiu, J. Sun, H. Farah, Safety performance boundary identification of highly automated vehicles: A surrogate model-based gradient descent searching approach, *IEEE Transactions on Intelligent Transportation Systems* (2022). doi:10.1109/TITS.2022.3191088.
- [22] S. Zhang, H. Peng, D. Zhao, H. E. Tseng, Accelerated evaluation of autonomous vehicles in the lane change scenario based on subset simulation technique, in: W. Zhang, A. M. Bayen, J. J. S. Medina, M. J. Barth (Eds.), *21st International Conference on Intelligent Transportation Systems, ITSC 2018, Maui, HI, USA, November 4-7, 2018, IEEE, 2018, pp. 3935–3940. doi:10.1109/ITSC.2018.8569800.*
- [23] R. Ben Abdesslem, S. Nejati, L. C. Briand, T. Stifter, Testing advanced driver assistance systems using multi-objective search and neural networks, in: *Proceedings of the 31st IEEE/ACM international conference on automated software engineering, 2016, pp. 63–74.*
- [24] W. Ding, B. Chen, B. Li, K. J. Eun, D. Zhao, Multimodal safety-critical scenarios generation for decision-making algorithms evaluation, *IEEE Robotics and Automation Letters* 6 (2) (2021) 1551–1558.
- [25] E. de Gelder, J. Paardekooper, Assessment of automated driving systems using real-life scenarios, in: *IEEE Intelligent Vehicles Symposium, IV 2017, Los Angeles, CA, USA, June 11-14, 2017, IEEE, 2017, pp. 589–594. doi:10.1109/IVS.2017.7995782.*
- [26] G. Bagschik, T. Menzel, M. Maurer, Ontology based

- scene creation for the development of automated vehicles, in: 2018 IEEE Intelligent Vehicles Symposium, IV 2018, Changshu, Suzhou, China, June 26-30, 2018, IEEE, 2018, pp. 1813–1820. doi:10.1109/IVS.2018.8500632.
- [27] B. Gangopadhyay, S. Khastgir, S. Dey, P. Dasgupta, G. Montana, P. A. Jennings, Identification of test cases for automated driving systems using bayesian optimization, in: 2019 IEEE Intelligent Transportation Systems Conference, ITSC 2019, Auckland, New Zealand, October 27-30, 2019, IEEE, 2019, pp. 1961–1967. doi:10.1109/ITSC.2019.8917103.
- [28] X. Zhang, S. Khastgir, P. A. Jennings, Scenario description language for automated driving systems: A two level abstraction approach, in: 2020 IEEE International Conference on Systems, Man, and Cybernetics, SMC 2020, Toronto, ON, Canada, October 11-14, 2020, IEEE, 2020, pp. 973–980. doi:10.1109/SMC42975.2020.9283417.
- [29] M. M. Haklay, P. Weber, Openstreetmap: User-generated street maps, IEEE Pervasive Comput. 7 (4) (2008) 12–18. doi:10.1109/MPRV.2008.80.
- [30] A. Diaz-Diaz, M. Ocaña, A. Llamazares, C. G. Huélamo, P. A. Revenga, L. M. Bergasa, HD maps: Exploiting opendrive potential for path planning and map monitoring, in: 2022 IEEE Intelligent Vehicles Symposium, IV 2022, Aachen, Germany, June 4-9, 2022, IEEE, 2022, pp. 1211–1217. doi:10.1109/IV51971.2022.9827297.
- [31] P. Bender, J. Ziegler, C. Stiller, Lanelets: Efficient map representation for autonomous driving, in: 2014 IEEE Intelligent Vehicles Symposium Proceedings, Dearborn, MI, USA, June 8-11, 2014, IEEE, 2014, pp. 420–425. doi:10.1109/IVS.2014.6856487.
- [32] M. Althoff, S. Urban, M. Koschi, Automatic conversion of road networks from opendrive to lanelets, in: 2018 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI), Singapore, Singapore, July 31 - Aug. 2, 2018, IEEE, 2018, pp. 157–162. doi:10.1109/SOLI.2018.8476801.
- [33] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, S. A. Seshia, Scenic: a language for scenario specification and scene generation, in: K. S. McKinley, K. Fisher (Eds.), Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019, ACM, 2019, pp. 63–78. doi:10.1145/3314221.3314633.
- [34] Measurable scenario description language reference, https://www.foretellix.com/wp-content/uploads/2020/07/M-SDL_LRM_OS.pdf (2020).
- [35] R. Queiroz, T. Berger, K. Czarnecki, Geoscenario: An open DSL for autonomous driving scenario representation, in: 2019 IEEE Intelligent Vehicles Symposium, IV 2019, Paris, France, June 9-12, 2019, IEEE, 2019, pp. 287–294. doi:10.1109/IVS.2019.8814107.
- [36] R. Majumdar, A. S. Mathur, M. Pirron, L. Stegner, D. Zufferey, Paracosm: A language and tool for testing autonomous driving systems, CoRR abs/1902.01084 (2019). arXiv:1902.01084.
- [37] Openscenario traffic scenario definition, <https://www.vector.com/int/en/products/products-a-z/software/dyna4/openscenario/>.
- [38] S. Ulbrich, T. Menzel, A. Reschka, F. Schuldt, M. Maurer, Defining and substantiating the terms scene, situation, and scenario for automated driving, in: IEEE 18th International Conference on Intelligent Transportation Systems, ITSC 2015, Gran Canaria, Spain, September 15-18, 2015, IEEE, 2015, pp. 982–988. doi:10.1109/ITSC.2015.164.
- [39] PEGASUS, Scenario description (2019). URL https://www.pegasusprojekt.de/files/tmpl/PDF-Symposium/04_Scenario-Description.pdf
- [40] Z. Zhong, G. Kaiser, B. Ray, Neural network guided evolutionary fuzzing for finding traffic violations of autonomous vehicles, IEEE Transactions on Software Engineering (2022).
- [41] R. B. Abdesslem, S. Nejati, L. C. Briand, T. Stifter, Testing vision-based control systems using learnable evolutionary algorithms, in: M. Chaudron, I. Crnkovic, M. Chechik, M. Harman (Eds.), Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018, ACM, 2018, pp. 1016–1026. doi:10.1145/3180155.3180160.
- [42] T. Akutsu, T. Tamura, A. A. Melkman, A. Takasu, On the complexity of finding a largest common subtree of bounded degree, Theoretical Computer Science 590 (2015) 2–16.
- [43] K. Deb, R. B. Agrawal, Simulated binary crossover for continuous search space, Complex Syst. 9 (2) (1995).
- [44] K. Deb, D. Deb, Analysing mutation schemes for real-parameter genetic algorithms, Int. J. Artif. Intell. Soft Comput. 4 (1) (2014) 1–28. doi:10.1504/IJAISC.2014.059280.
- [45] J. Chen, M. Yan, Z. Wang, Y. Kang, Z. Wu, Deep neural network test coverage: How far are we?, arXiv preprint arXiv:2010.04946 (2020).
- [46] Y. Feng, Q. Shi, X. Gao, J. Wan, C. Fang, Z. Chen, Deepgini: prioritizing massive tests to enhance the robustness of deep neural networks, in: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2020, pp. 177–188.
- [47] C. E. Shannon, A mathematical theory of communication, The Bell System Technical Journal 27 (3) (1948) 379–423. doi:10.1002/j.1538-7305.1948.tb01338.x.
- [48] Y. Tian, K. Pei, S. Jana, B. Ray, Deeptest: automated testing of deep-neural-network-driven autonomous cars, in: M. Chaudron, I. Crnkovic, M. Chechik, M. Harman (Eds.), Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018, ACM, 2018, pp. 303–314. doi:10.1145/3180155.3180220.

- [49] M. Zhang, Y. Zhang, L. Zhang, C. Liu, S. Khurshid, Deeproad: Gan-based metamorphic testing and input validation framework for autonomous driving systems, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, 2018, pp. 132–142.
- [50] H. Zhou, W. Li, Z. Kong, J. Guo, Y. Zhang, B. Yu, L. Zhang, C. Liu, Deepbillboard: systematic physical-world testing of autonomous driving systems, in: G. Rothermel, D. Bae (Eds.), ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020, ACM, 2020, pp. 347–358. doi:10.1145/3377811.3380422.
- [51] T. Y. Chen, F. Kuo, H. Liu, P. Poon, D. Towey, T. H. Tse, Z. Q. Zhou, Metamorphic testing: A review of challenges and opportunities, ACM Comput. Surv. 51 (1) (2018) 4:1–4:27. doi:10.1145/3143561.
- [52] K. Pei, Y. Cao, J. Yang, S. Jana, Deepxplore: Automated whitebox testing of deep learning systems, in: Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28–31, 2017, ACM, 2017, pp. 1–18. doi:10.1145/3132747.3132785.
- [53] NVIDIA, Nvidia-autopilot-keras, <https://github.com/Observer07/Nvidia-Autopilot-Keras> (2016).
- [54] J. Gildenblat, Visualizations for understanding the regressed wheel steering angle for self driving cars, <https://github.com/jacobgil/keras-steering-angle-visualizations> (2016).
- [55] A. Staravoi, Behavioral cloning: end-to-end learning for self-driving cars, <https://github.com/navoshta/behavioral-cloning> (2016).
- [56] Udacity, Using deep learning to predict steering angles, <https://medium.com/udacity/challenge-2-using-deep-learning-to-predict-steering-angles-f42004a36ff3> (2016).
- [57] T. Y. Chen, H. Leung, I. K. Mak, Adaptive random testing, in: M. J. Maher (Ed.), Advances in Computer Science - ASIAN 2004, Higher-Level Decision Making, 9th Asian Computing Science Conference, Dedicated to Jean-Louis Lassez on the Occasion of His 5th Cycle Birthday, Chiang Mai, Thailand, December 8–10, 2004, Proceedings, Vol. 3321 of Lecture Notes in Computer Science, Springer, 2004, pp. 320–329. doi:10.1007/978-3-540-30502-6_23.
- [58] T. Chen, R. Merkel, P. Wong, G. Eddy, Adaptive random testing through dynamic partitioning, in: Fourth International Conference on Quality Software, 2004. QSIC 2004. Proceedings., 2004, pp. 79–86. doi:10.1109/QSIC.2004.1357947.
- [59] Y. Zhang, Q. Lin, J. Wang, S. Verwer, J. M. Dolan, Lane-change intention estimation for car-following control in autonomous driving, IEEE Trans. Intell. Veh. 3 (3) (2018) 276–286. doi:10.1109/TIV.2018.2843178.
- [60] F. Hauer, T. Schmidt, B. Holzmüller, A. Pretschner, Did we test all scenarios for automated and autonomous driving systems?, in: 2019 IEEE Intelligent Transportation Systems Conference, ITSC 2019, Auckland, New Zealand, October 27–30, 2019, IEEE, 2019, pp. 2950–2955. doi:10.1109/ITSC.2019.8917326.
- [61] G. E. Mullins, P. G. Stankiewicz, R. C. Hawthorne, S. K. Gupta, Adaptive generation of challenging scenarios for testing and evaluation of autonomous vehicles, Journal of Systems and Software 137 (2018) 197–215.
- [62] A. Guo, Y. Feng, Z. Chen, Lirtest: augmenting lidar point clouds for automated testing of autonomous driving systems, in: S. Ryu, Y. Smaragdakis (Eds.), ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, South Korea, July 18 - 22, 2022, ACM, 2022, pp. 480–492. doi:10.1145/3533767.3534397.
- [63] Z. Zhong, Z. Hu, S. Guo, X. Zhang, Z. Zhong, B. Ray, Detecting multi-sensor fusion errors in advanced driver-assistance systems, in: S. Ryu, Y. Smaragdakis (Eds.), ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, South Korea, July 18 - 22, 2022, ACM, 2022, pp. 493–505. doi:10.1145/3533767.3534223.
- [64] S. Masuda, H. Nakamura, K. Kajitani, Rule-based searching for collision test cases of autonomous vehicles simulation, IET Intelligent Transport Systems 12 (9) (2018) 1088–1095. doi:https://doi.org/10.1049/iet-its.2018.5335.
- [65] A. Gambi, M. Müller, G. Fraser, Automatically testing self-driving cars with search-based procedural content generation, in: D. Zhang, A. Möller (Eds.), Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15–19, 2019, ACM, 2019, pp. 318–328. doi:10.1145/3293882.3330566.
- [66] S. Kim, M. Liu, J. J. Rhee, Y. Jeon, Y. Kwon, C. H. Kim, Drivefuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, 2022, pp. 1753–1767.
- [67] H. Liang, X. Pei, X. Jia, W. Shen, J. Zhang, Fuzzing: State of the art, IEEE Trans. Reliab. 67 (3) (2018) 1199–1218. doi:10.1109/TR.2018.2834476.
- [68] J. Li, B. Zhao, C. Zhang, Fuzzing: a survey, Cybersec. 1 (1) (2018) 6. doi:10.1186/s42400-018-0002-y.
- [69] J. Guo, Y. Jiang, Y. Zhao, Q. Chen, J. Sun, Dlfuzz: Differential fuzzing testing of deep learning systems, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2018, pp. 739–743.
- [70] Z. Hu, J. Shi, Y. Huang, J. Xiong, X. Bu, Ganfuzz: a gan-based industrial network protocol fuzzing framework, in: D. R. Kaeli, M. Pericàs (Eds.), Proceedings of the 15th ACM International Conference on Computing Frontiers, CF 2018, Ischia, Italy, May 08–10, 2018, ACM, 2018, pp. 138–145. doi:10.1145/3203217.3203241.
- [71] V. Atlidakis, R. Geambasu, P. Godefroid, M. Polishchuk,

B. Ray, Pythia: Grammar-based fuzzing of REST apis with coverage-guided feedback and learning-based mutations, CoRR abs/2005.11498 (2020). [arXiv:2005.11498](https://arxiv.org/abs/2005.11498).

- [72] M. Eberlein, Y. Noller, T. Vogel, L. Grunske, Evolutionary grammar-based fuzzing, in: A. Aleti, A. Panichella (Eds.), Search-Based Software Engineering - 12th International Symposium, SSBSE 2020, Bari, Italy, October 7-8, 2020, Proceedings, Vol. 12420 of Lecture Notes in Computer Science, Springer, 2020, pp. 105–120. [doi: 10.1007/978-3-030-59762-7_8](https://doi.org/10.1007/978-3-030-59762-7_8).